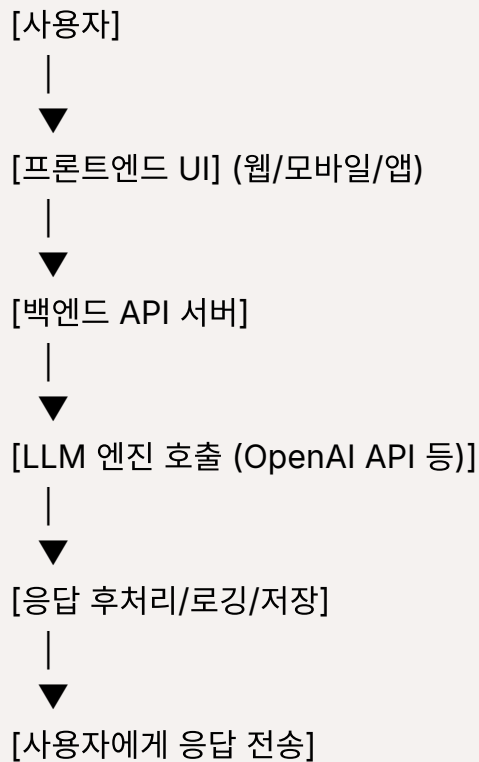


How a Prompt Travels through an LLM Service

1. 전체 구조 개요: LLM 서비스 구조



2. 구성 요소별 설명

① 사용자 인터페이스 (UI)

- 보통 웹브라우저나 앱에서 사용자가 **프롬프트를 입력**합니다.
- 입력은 HTTP(S) 요청으로 백엔드 서버에 전달됩니다.
- 예: ChatGPT의 경우 chat.openai.com 에서 채팅 인터페이스 제공

② API 서버 (백엔드)

- 사용자 요청을 수신하여 **검증, 전처리**를 수행합니다.
- OpenAI나 Hugging Face 등의 **LLM API로 전달**합니다.
- 인증 (API Key, 토큰), 사용자별 설정 관리, 사용량 제한 등 포함

③ LLM 엔진 (모델 서버)

- 프롬프트를 받아 모델에 입력하고, **토큰 단위로 응답 생성**합니다.
- OpenAI의 경우, `gpt-4o-mini`, `gpt-3.5-turbo`, `gpt-4o` 등 다양한 모델이 존재
- 내부적으로는 다음 과정을 거칩니다:
 1. 토큰나이징 (Tokenization)
 2. 입력 임베딩 → Transformer 네트워크 처리
 3. 다음 토큰 생성 (Autoregressive 방식)
 4. 토큰을 반복 생성하여 응답 구성

④ 후처리 및 보안 제어

- 생성된 응답을 필터링하거나 수정합니다.
- 예시: 유해 콘텐츠 감지, 사용자 정책 위반 여부 검사
- 로그 기록, 분석 시스템 연동 등 포함

⑤ 사용자에게 응답 반환

- 생성된 텍스트가 다시 UI로 전송되어 사용자가 결과를 확인

3. LLM 응답 흐름 상세 (ChatGPT 예시 기반)

- (1) 사용자가 "오늘 날씨 어때?" 입력 →
- (2) UI → API 서버로 HTTP 요청 전송
- (3) API 서버는 메시지를 구성 (시스템 메시지 + 사용자 입력 포함)
- (4) OpenAI LLM API 호출: `client.chat.completions.create(...)`
- (5) 모델이 응답 생성: `{"role": "assistant", "content": "오늘은 맑고 따뜻한 날씨입니다."}`
- (6) API 서버는 응답을 사용자 세션에 저장하고 UI에 전달
- (7) 사용자가 화면에서 응답 확인

ChatGPT와 같은 LLM 서비스는 복잡한 딥러닝 모델을 추상화하여, 사용자가 **자연어로 간단히 입력하고 즉시 결과를 받는 서비스 구조**입니다.

이 전체 흐름은 마치 **질문 → 이해 → 추론 → 대답**이라는 인간 사고 과정과 유사하게 설계되어 있습니다.

4. 개발자 관점: OpenAI API 호출 예시

```
from openai import OpenAI
client = OpenAI()
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "당신은 친절한 비서입니다."},
        {"role": "user", "content": "오늘 할 일 목록을 정리해줘"}
    ],
    temperature=0.7)
print(response.choices[0].message.content)
```

- 시스템 메시지는 LLM에게 역할을 정의해 줍니다.
- 유저 입력을 기반으로 모델이 응답을 생성합니다.
- 이처럼 **프롬프트 → 응답 생성 → 반환**의 구조로 작동합니다.

5. 부가 요소

구성 요소	설명
벡터 DB	RAG에서 검색 기반 응답을 위한 문서 임베딩 저장소
Function Calling	도구 호출을 통한 계산/검색/실행 가능
안 필터링	민감정보/정책 위반 프롬프트 필터링
로깅/모니터링	응답 저장, 품질 분석, 트래픽 분석